ETS

PEMROGRAMAN ROBOTIKA DENGAN KIT iMCLab MENGGUNAKAN PYTHON

Dosen Pengampu:
Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:
Moh Avin Dharma Wijaya (23081010243)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

BAB I PERINTAH DAN KODE PROGRAM 1

1.1 Ragam masukan pada antarmuka Python 3

1.2 Penyalinan sel program 4

BAB II HASIL DAN PEMBAHASAN 10

2.1 Penentuan port dan persiapan sesi 11

2.2 Pengujian perintah penyalaan dan pemadaman motor 13

2.3 Keadaan perangkat pada kondisi motor mati 14

2.4 Keadaan perangkat pada kondisi motor berputar 15

2.5 Pengujian kenaikan kecepatan bertahap 16

2.6 Pengujian penurunan kecepatan bertahap 17

2.7 Pengujian penolakan perintah tidak sah 18

2.8 Rangkuman hasil pengujian 21

2.9 Dokumentasi Praktikum 23

BAB III PENUTUP 28

3.1 Kesimpulan 29

3.2 Saran 30

DAFTAR GAMBAR

Gambar 1. Sel program dan penelusuran port serial pada Jupyter Notebook 12
Gambar 2. Keluaran perintah penyalaan dan pemadaman motor 14
Gambar 3. Diagram rangkaian perangkat pada kondisi motor mati 14
Gambar 4. Diagram rangkaian perangkat pada kondisi motor berputar 16
Gambar 5. Keluaran perintah kenaikan kecepatan sebanyak dua kali 17
Gambar 6. Keluaran perintah penurunan kecepatan 18
Gambar 7. Penolakan perintah PWM:220 oleh program Python 19
Gambar 8. Penolakan masukan kosong oleh validasi program 20
Gambar 9. Kit iMCLab saat dikendalikan melalui program Python pada Jupyter Notebook 25
Gambar 10. Kit iMCLab dan roda piringan selama pengujian antarmuka Python27

BAB I PERINTAH DAN KODE PROGRAM

Pemindahan kendali motor DC dari Serial Monitor Arduino IDE ke Jupyter Notebook mengubah peran kedua pihak yang berkomunikasi. Berkas `Tes_IMCLab_naik_turun.ipynb` bertindak sebagai sisi host yang memegang inisiatif: ia yang membuka kanal, menyusun teks perintah, mengirimkannya, kemudian mengumpulkan balasan untuk ditampilkan. Firmware `imclab_naik_turun.ino` pada board ESP32 tetap memegang peran sebagai pelaksana di sisi perangkat, yaitu menafsirkan perintah menjadi nilai `dutyCycle` dan menuliskannya ke kanal PWM melalui `ledcWrite`.

Kanal yang dipakai adalah serial USB dengan kecepatan 115200 baud, dibuka melalui pustaka `pyserial`. Perbedaan mendasar dibanding cara kerja Serial Monitor terletak pada penentuan port. Alih-alih menyebutkan nama port secara manual, program menitipkan proses pencarian port kepada fungsi `deteksi_port_esp32()`, yang menyusun peringkat seluruh port serial yang ada berdasarkan ciri khas perangkat ESP32.

1.1 Ragam masukan pada antarmuka Python

Antarmuka interaktif mengenali enam masukan. Tabel berikut merangkum masukan tersebut beserta fungsi yang dijalankan dan rentetan yang benar-benar dikirimkan ke ESP32.

No	Masukan	Fungsi yang dipanggil	Rentetan terkirim	Akibat di perangkat
1	1	motor_on(ser)	1	Motor menyala dengan dutyCycle terakhir; bila bernilai 0 dipakai nilai awal 200
2	0	motor_off(ser)	0	dutyCycle dijadikan 0 sehingga motor berhenti
3	+	kecepatan_naik(ser)	+	dutyCycle bertambah DUTY_STEP sebesar 10, dibatasi DUTY_MAX 255
4	-	kecepatan_turun(ser)	-	dutyCycle berkurang DUTY_STEP sebesar 10, dibatasi DUTY_MIN 0
5	p<val>	set_pwm(ser, int(cmd[1:]))	PWM:<val>	Fungsi setDuty() pada firmware menetapkan dutyCycle sesuai angka
6	q	keluar dari while True lalu blok finally	—	motor_off() dipanggil lebih dahulu, kemudian port serial ditutup

Dua hal perlu dicatat dari tabel tersebut. Pertama, masukan `p<val>` pada

kolom kedua adalah bentuk yang diketikkan pengguna, sedangkan kolom ketiga adalah

bentuk yang beredar di jalur serial; fungsi `set_pwm()` yang menjembatani keduanya. Kedua, pesan bantuan yang dicetak program menyatakan contoh `p150` untuk perintah ini, sehingga nilai 150 pada contoh tersebut adalah angka duty, bukan bagian dari nama perintah.

1.2 Penyalinan sel program

Seluruh isi sel kode pada `Tes_IMCLab_naik_turun.ipynb` disalin berikut ini tanpa satu pun perubahan, dari baris komentar pembuka sampai pemanggilan penutupan koneksi serial.

```
# ============================================================
# Program Kontrol Motor ESP32 via Serial (Auto-Detect Port)
# Fitur: ON/OFF + Naik/Turun kecepatan (PWM) - Mode Interaktif
# ============================================================

import serial
import serial.tools.list_ports
import sys
import time

# ==========================================
# KONFIGURASI
# ==========================================
BAUDRATE = 115200

ESP32_KEYWORDS = [
    "cp210",
    "cp2102",
    "cp210x",
    "ch340",
    "ch341",
    "ch34x",
    "ftdi",
    "ft232",
    "usb-serial",
    "usb serial",
    "usb uart",
```

```
    "uart bridge",
    "silicon labs",
    "espressif",
    "esp32",
]

ESP32_VID_PID = {
    (0x10C4, 0xEA60),
    (0x10C4, 0xEA70),
    (0x1A86, 0x7523),
    (0x1A86, 0x55D4),
    (0x0403, 0x6001),
    (0x0403, 0x6015),
    (0x303A, 0x1001),
}

DUTY_STEP = 10  # selisih kenaikan/penurunan di sisi Python (informatif)



# ============================================
# Auto-detect port ESP32
# ============================================
def list_serial_ports():
  ports = serial.tools.list_ports.comports()
  if not ports:
    print("Tidak ada port serial yang terdeteksi.")
    return []
  print("Port serial yang tersedia:")
  for p in ports:
    vid_pid = ""
    if p.vid is not None and p.pid is not None:
      vid_pid = f" [VID:PID={p.vid:04X}:{p.pid:04X}]"
    print(f"  - {p.device} : {p.description}{vid_pid}")
  return list(ports)



def deteksi_port_esp32(verbose=True):
  ports = serial.tools.list_ports.comports()
  if not ports:
    if verbose:
      print("[!] Tidak ada port serial yang terdeteksi.")
    return None

  kandidat = []
  for p in ports:
    desc = (p.description or "").lower()
    mfg = (p.manufacturer or "").lower()
    hwid = (p.hwid or "").lower()
    teks = f"{desc} {mfg} {hwid}"

    skor = 0
    if p.vid is not None and p.pid is not None:
      if (p.vid, p.pid) in ESP32_VID_PID:
```

```
        skor += 100
    for kw in ESP32_KEYWORDS:
      if kw in teks:
        skor += 50
        break
    if "bluetooth" in teks:
      skor -= 200

    if skor > 0:
      kandidat.append((skor, p))

  if not kandidat:
    non_bt = [
        p for p in ports if "bluetooth" not in (p.description or "").lower()
    ]
    if len(non_bt) == 1:
      if verbose:
        print(f"[i] Hanya ada 1 port non-Bluetooth: {non_bt[0].device}")
      return non_bt[0].device
    if verbose:
      print("[!] Port ESP32 tidak dapat dideteksi otomatis.")
    return None

  kandidat.sort(key=lambda x: x[0], reverse=True)
  if verbose:
    print("Kandidat port ESP32 (skor tertinggi = paling mungkin):")
    for skor, p in kandidat:
      print(f"  [{skor:>4}] {p.device} : {p.description}")
  return kandidat[0][1].device


# ============================================
# Koneksi serial
# ============================================
def buka_koneksi(port, baudrate=115200):
  try:
    ser = serial.Serial(port, baudrate, timeout=1)
    time.sleep(2)
    print(f"Terhubung ke ESP32 di {port} @ {baudrate} baud")
    return ser
  except serial.SerialException as e:
    print(f"Gagal membuka port {port}: {e}")
    return None


# ============================================
# Kirim perintah & baca balasan
# ============================================
def kirim_perintah(ser, perintah: str):
  """Mengirim perintah ke ESP32 (diakhiri newline).

  Perintah valid: '0', '1', '+', '-', 'PWM:xxx'
  """
```

```
  ser.write((perintah + "\n").encode("utf-8"))
  ser.flush()
  time.sleep(0.15)

  # Baca semua baris balasan yang tersedia
  while ser.in_waiting > 0:
    balasan = ser.readline().decode("utf-8", errors="ignore").strip()
    if balasan:
      print(f"ESP32: {balasan}")
    time.sleep(0.05)


# Wrapper perintah
def motor_on(ser):
  print(">> Motor ON")
  kirim_perintah(ser, "1")


def motor_off(ser):
  print(">> Motor OFF")
  kirim_perintah(ser, "0")


def kecepatan_naik(ser, step=DUTY_STEP):
  """Naikkan kecepatan motor sebanyak `step` (default 10)."""
  print(f">> Kecepatan +{step}")
  kirim_perintah(ser, "+")


def kecepatan_turun(ser, step=DUTY_STEP):
  """Turunkan kecepatan motor sebanyak `step` (default 10)."""
  print(f">> Kecepatan -{step}")
  kirim_perintah(ser, "-")


def set_pwm(ser, duty: int):
  """Set duty cycle langsung (0-255)."""
  duty = max(0, min(255, int(duty)))
  print(f">> Set PWM = {duty}")
  kirim_perintah(ser, f"PWM:{duty}")


# ============================================
# PROGRAM UTAMA
# ============================================
if __name__ == "__main__":
  print("=" * 50)
  list_serial_ports()
  print("=" * 50)

  PORT = deteksi_port_esp32()
```

```
if PORT is None:
  print("Program dihentikan: port ESP32 tidak ditemukan.")
  sys.exit(1)
print(f"\n>> Port ESP32 terpilih: {PORT}")

esp32 = buka_koneksi(PORT, BAUDRATE)

if esp32 is None:
  print("Koneksi gagal. Periksa kembali PORT dan kabel USB.")
else:
  try:
    # ---- Mode Interaktif Langsung Aktif ----
    print("\n" + "=" * 50)
    print("MODE KONTROL MOTOR INTERAKTIF SIAP")
    print("=" * 50)
    print("  '1'     = Motor ON")
    print("  '0'     = Motor OFF")
    print("  '+'     = Naikkan kecepatan (+10)")
    print("  '-'     = Turunkan kecepatan (-10)")
    print("  'p<val>'= Set PWM langsung (contoh: p150)")
    print("  'q'     = Keluar / Exit")
    print("=" * 50)

    while True:
      cmd = input("\nMasukkan Perintah: ").strip()

      if cmd.lower() == "q":
        print("Keluar dari program...")
        break
      elif cmd == "1":
        motor_on(esp32)
      elif cmd == "0":
        motor_off(esp32)
      elif cmd == "+":
        kecepatan_naik(esp32)
      elif cmd == "-":
        kecepatan_turun(esp32)
      elif cmd.lower().startswith("p") and cmd[1:].isdigit():
        set_pwm(esp32, int(cmd[1:]))
      else:
        print(
            "[!] Perintah tidak valid! Gunakan: 1, 0, +, -, p<angka> (misal:"
            " p150), atau q."
        )

  except KeyboardInterrupt:
    print("\n[!] Dihentikan oleh pengguna (Ctrl+C).")

  finally:
    try:
      motor_off(esp32)
    except Exception:
      pass
    esp32.close()
```

```
print("Koneksi serial ditutup. Sampai jumpa!")
```

BAB II HASIL DAN PEMBAHASAN

2.1 Penentuan port dan persiapan sesi

Port serial yang layak dipakai ditentukan sepenuhnya oleh fungsi `deteksi_port_esp32()`, sehingga bagian ini bertujuan memastikan mekanisme penilaian tersebut bekerja pada komputer yang dipakai. Sel program dijalankan pada Jupyter Notebook dengan kernel Python 3 (ipykernel).

Fungsi tersebut membentuk satu rentetan teks gabungan dari tiga atribut tiap port, yaitu `desc` dari `p.description`, `mfg` dari `p.manufacturer`, dan `hwid` dari `p.hwid`. Rentetan itulah yang dicari terhadap daftar `ESP32_KEYWORDS`.

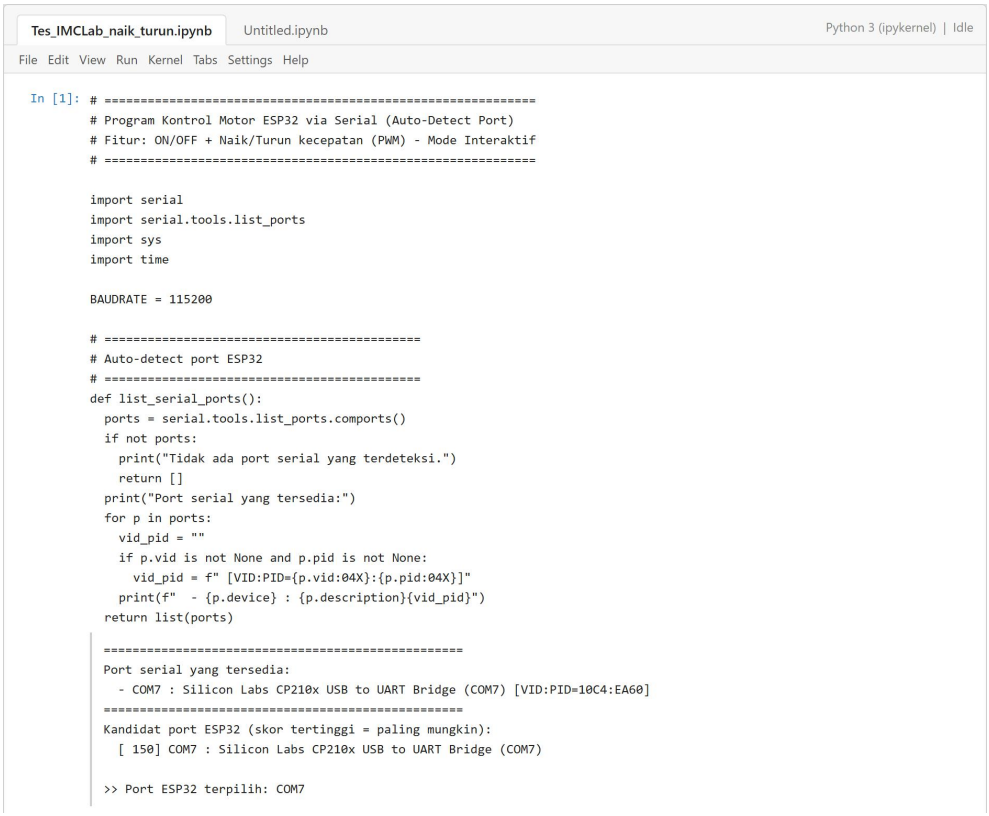
Skema skornya bertingkat:

Ciri yang ditemukan	**Nilai skor**
Pasangan (p.vid, p.pid) ada di dalam himpunan ESP32_VID_PID	+100
Salah satu kata kunci pada ESP32_KEYWORDS terkandung dalam rentetan teks	+50
Kata "bluetooth" terkandung dalam rentetan teks	−200

Hanya port dengan `skor > 0` yang masuk daftar `kandidat`, lalu daftar tersebut diurutkan menurun dan nilai teratas diambil sebagai hasil.

Pada sesi yang tercatat, hanya satu port yang tersedia, yaitu COM7 dengan deskripsi Silicon Labs CP210x USB to UART Bridge dan identitas VID:PID = 10C4:EA60. Pasangan tersebut tergolong ke dalam `ESP32_VID_PID`, sehingga skornya +100, ditambah +50 dari kecocokan kata kunci deskripsi, menghasilkan skor akhir 150. COM7 kemudian dipilih, koneksi dibuka pada

115200 baud, dan menu mode kontrol motor interaktif tercetak.



Gambar 1. Sel program dan penelusuran port serial pada Jupyter Notebook

Perlu ditegaskan bahwa penalti −200 pada baris `if "bluetooth" in teks: skor -= 200`

bukan sekadar pemberat biasa. Sebuah port Bluetooth virtual yang kebetulan mengandung kata "uart" atau "usb serial" pada deskripsinya akan memperoleh +100 dari kata kunci, tetapi masih berakhir pada −100 dan karena itu gagal melewati syarat `skor > 0`. Penalti sebesar itu juga membuat port Bluetooth mustahil menyusul port USB mana pun yang berhasil mencetak skor 100 sampai 150. Dengan kata lain, angka 200 dipilih agar penalti lebih besar daripada kombinasi skor tertinggi yang masih mungkin dicapai hanya dari pencocokan kata kunci.

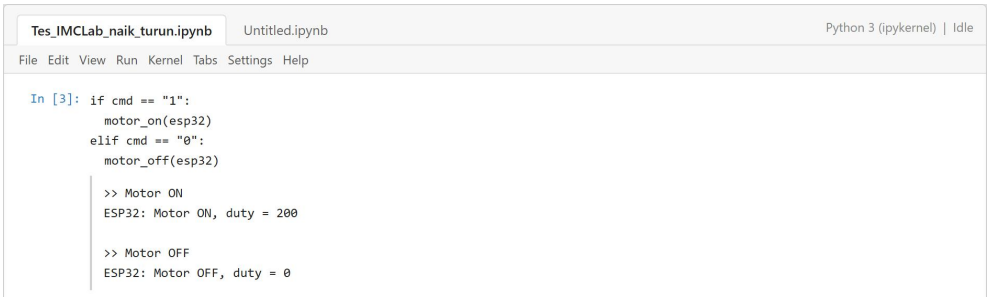
Mekanisme cadangan baru dijalankan bila daftar `kandidat` kosong sama sekali, yaitu ketika seluruh port tersaring keluar oleh syarat `skor > 0`. Dalam keadaan itu fungsi menyusun ulang daftar port non-Bluetooth dengan cara menyaring `(p.description or "").lower()`, dan bila panjang daftar itu tepat satu, port tunggal tersebut langsung dikembalikan tanpa melalui penghitungan skor. Pola ini masuk akal untuk pengujian di ruang praktikum, karena board yang terhubung tunggal umumnya memang target yang benar meski chip USB-serial-nya belum terdaftar pada `ESP32_VID_PID`. Bila tersisa lebih dari satu port non-Bluetooth, fungsi memilih berhenti dan mengembalikan `None`, yang akan memicu `sys.exit(1)` pada program utama.

2.2 Pengujian perintah penyalaan dan pemadaman motor

Tujuan pengujian ini adalah memastikan dua keadaan paling dasar pada motor, yakni berputar dan berhenti, dapat dicapai dari sisi host sekaligus memastikan jalur balasan dari ESP32 benar-benar terbaca. Masukan yang dicoba adalah `1` kemudian `0` pada kotak input interaktif.

Setiap masukan menghasilkan dua baris keluaran dengan asal yang berbeda. Baris `>> Motor ON` dan `>> Motor OFF` dicetak oleh fungsi `motor_on()` dan `motor_off()` di sisi Python. Baris `ESP32: Motor ON, duty = 200` dan `ESP32: Motor OFF, duty = 0` merupakan teks yang datang dari firmware, dibaca oleh `ser.readline()` di dalam perulangan pada `kirim_perintah()`, lalu ditampilkan dengan awalan `ESP32:`. Kehadiran kedua kelompok baris ini menjadi bukti bahwa kanal serial berjalan dua arah.

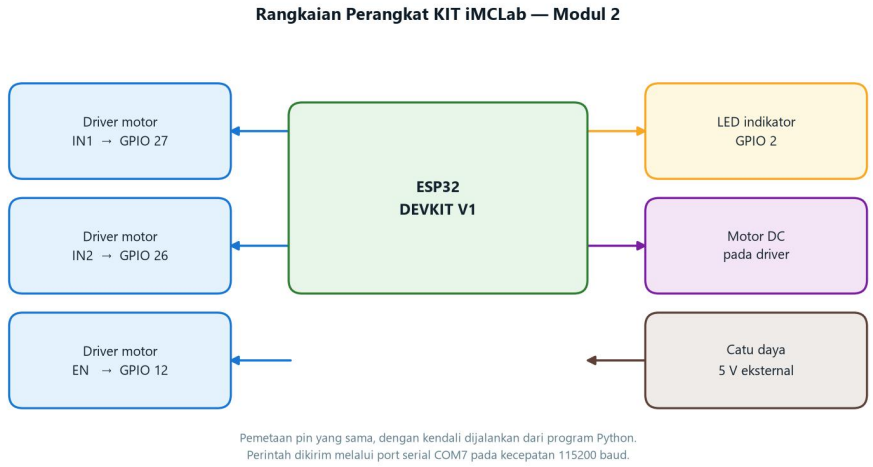
Angka `duty = 200` pada balasan penyalaan sesuai dengan perilaku fungsi `MotorOn()` pada firmware: bila `dutyCycle` bernilai 0, nilai itu diganti terlebih dahulu dengan 200 sebelum `applyMotor()` dipanggil. Sebaliknya, `MotorOff()` selalu menuliskan nilai 0 ke `dutyCycle` lebih dahulu, sehingga balasan yang terlihat adalah `duty = 0` tanpa bergantung pada nilai sebelumnya.



Gambar 2. Keluaran perintah penyalaan dan pemadaman motor

2.3 Keadaan perangkat pada kondisi motor mati

Bagian ini menelaah kecocokan antara laporan numerik pada layar dan keadaan perangkat keras pada saat `dutyCycle` bernilai 0, yaitu keadaan yang dihasilkan perintah `0` maupun keadaan awal setelah `setup()` selesai dijalankan.

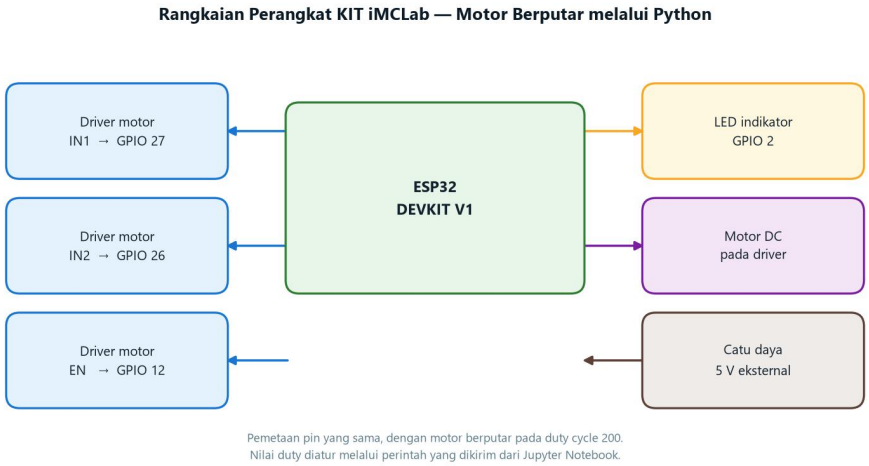


Gambar 3. Diagram rangkaian perangkat pada kondisi motor mati

Ketika `applyMotor()` dipanggil dengan `dutyCycle` sama dengan 0, perintah `ledcWrite(pwmChannel, dutyCycle)` mengirimkan lebar pulsa nol ke pin `enable1Pin` pada GPIO 12. Tanpa lebar pulsa, driver motor tidak menerima tegangan rata-rata yang cukup untuk memutar poros, sehingga roda berhenti. Pada kondisi yang sama, baris `digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW)` mengevaluasi perbandingan menjadi salah dan menurunkan tegangan pada `ledPin` di GPIO 2, sehingga LED indikator biru padam. Diagram rangkaian pada Gambar 3 memperlihatkan jalur sinyal pada kondisi tersebut, yaitu kanal PWM berada pada nilai nol sehingga tidak ada daya yang disalurkan ke motor. Pin `motor1Pin1` tetap bernilai HIGH dan `motor1Pin2` tetap LOW, jadi yang memutus gerak motor adalah nilai duty, bukan perubahan polaritas.

2.4 Keadaan perangkat pada kondisi motor berputar

Bagian ini menelaah sisi kebalikannya, yaitu keadaan perangkat ketika `dutyCycle` berada pada nilai 200 dan motor benar-benar berputar setelah perintah `1` dikirim dari notebook.



Gambar 4. Diagram rangkaian perangkat pada kondisi motor berputar

Dengan `dutyCycle` bernilai 200, `ledcWrite()` membangkitkan pulsa pada kanal PWM `pwmChannel` 0 dengan frekuensi 30000 Hz dan resolusi 8 bit, sehingga rasio duty efektifnya sekitar 78 persen dari rentang 0 sampai 255. Rasio inilah yang membuat poros motor berputar pada kecepatan yang cukup tinggi.

Diagram rangkaian pada Gambar 4 memperlihatkan jalur sinyal yang aktif, dan hal yang hendak ditunjukkan adalah kesesuaian tiga penanda sekaligus: kanal PWM yang aktif, LED indikator biru pada GPIO 2 yang menyala, dan nilai `duty = 200` pada keluaran notebook. Kesamaan ketiganya menunjukkan bahwa angka yang dicetak

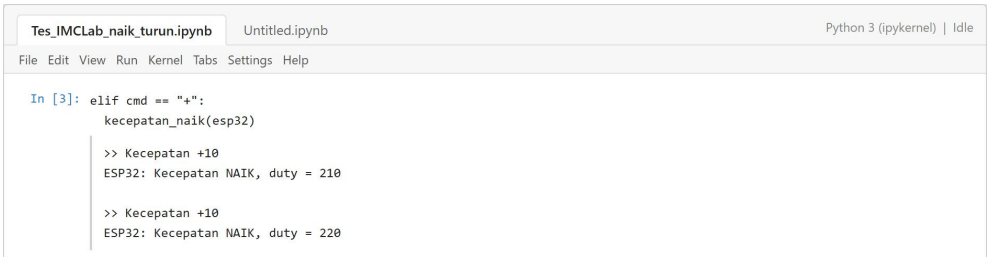
di layar bukan sekadar teks, melainkan pantulan dari penerapan nilai pada pin perangkat.

2.5 Pengujian kenaikan kecepatan bertahap

Perubahan kecepatan secara bertahap diuji untuk memeriksa apakah kenaikan kecepatan berjalan sebagai langkah kecil yang dapat diulang, bukan sebagai satu

lompatan nilai. Perintah + dimasukkan dua kali berturut-turut ketika `dutyCycle` berada pada nilai 200.

Pembagian tanggung jawab antara kedua sisi terlihat jelas di sini. Fungsi `kecepatan_naik()` pada sisi Python hanya mencetak `>> Kecepatan +10` lalu mengirimkan satu karakter +; ia tidak menyertakan nilai duty di dalam pesan perintah. Selisih angka yang tercatat pada balasan berasal dari baris `dutyCycle += DUTY_STEP` di dalam fungsi `naikkanKecepatan()` pada firmware. Karena itu, konstanta `DUTY_STEP` yang dideklarasikan pada sisi Python bersifat informatif: nilainya hanya dipakai untuk menyusun teks pesan melalui parameter `step`, sementara nilai yang benar-benar menentukan besar kenaikan adalah `DUTY_STEP` milik firmware yang bernilai 10. Balasan yang terbaca adalah `Kecepatan NAIK, duty = 210` pada penekanan pertama dan `duty = 220` pada penekanan kedua.

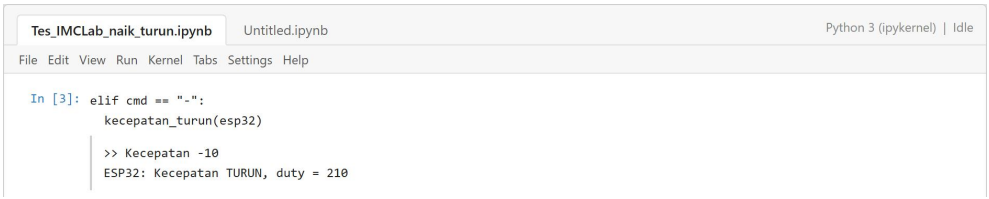


Gambar 5. Keluaran perintah kenaikan kecepatan sebanyak dua kali

2.6 Pengujian penurunan kecepatan bertahap

Penurunan diuji untuk memeriksa apakah langkah perubahan bersifat simetris dengan kenaikan, karena keduanya memakai besar langkah yang sama. Perintah - dimasukkan satu kali sesudah `dutyCycle` mencapai 220.

Fungsi `kecepatan_turun()` mencetak `>> Kecepatan -10` dan mengirimkan karakter `-`. Pada firmware, fungsi `turunkanKecepatan()` menjalankan `dutyCycle -= DUTY_STEP` lalu membatasi hasilnya agar tidak turun di bawah `DUTY_MIN` yang bernilai 0. Balasan yang terbaca adalah `Kecepatan TURUN, duty = 210`, tepat 10 satuan lebih rendah dari nilai sebelumnya. Bila seluruh sesi pengujian dirangkai, urutan nilai duty yang tercatat adalah 200 → 210 → 220 → 210. Urutan itu menunjukkan bahwa `dutyCycle` pada firmware berperan sebagai keadaan yang tersimpan selama sesi berlangsung; setiap perintah dihitung dari nilai terakhir, bukan dari nilai awal 200.



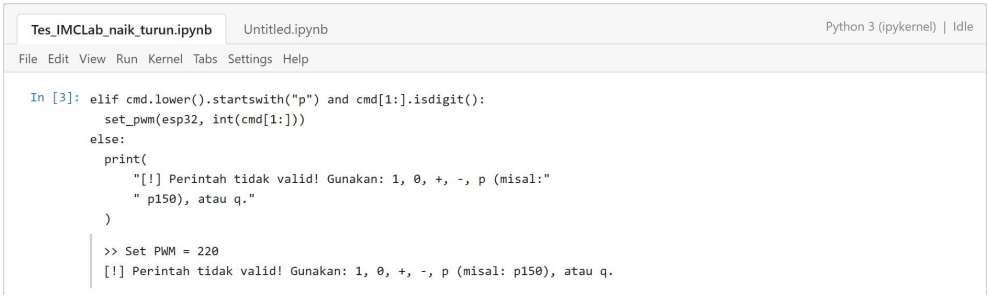
Gambar 6. Keluaran perintah penurunan kecepatan

2.7 Pengujian penolakan perintah tidak sah

Dua jenis masukan tidak sah dicoba untuk memeriksa ketahanan perulangan interaktif: masukan `PWM:220` dan masukan kosong. Keduanya penting karena keluaran repo pada modul ini justru mencatat penolakan `PWM:220`, padahal perintah itu sah pada Serial Monitor Arduino IDE dan diterima oleh firmware `imclab_naik_turun.ino`.

Sumber penolakan tersebut dapat ditelusuri pada satu baris percabangan, `elif cmd.lower().startswith("p") and cmd[1:].isdigit():`. Agar cabang ini terbuka, masukan harus memenuhi dua syarat sekaligus: huruf pertamanya `p`

(setelah dinormalkan menjadi huruf kecil) dan sisa karakternya seluruhnya berupa angka. Masukan `PWM:220` gagal pada syarat pertama karena huruf pertamanya, setelah dinormalkan, adalah `p` diikuti huruf `w` yang bukan angka. Yang benar-benar dijalankan cabang tersebut adalah bentuk `p<val>`, misalnya `p220`, yang kemudian diolah oleh `set_pwm()` menjadi rentetan `PWM:220` sebelum dikirim ke ESP32.
Di sinilah letak ketidaksesuaian antarmuka yang perlu dicatat secara jujur. Petunjuk pada program menyatakan bentuk perintah `p<val>` dengan contoh `p150`. Docstring fungsi `kirim_perintah()` menyatakan sebaliknya, yaitu bahwa perintah valid adalah `'0'`, `'1'`, `'+'`, `'-'`, dan `'PWM:xxx'`. Keduanya bicara tentang lapisan yang berbeda: `PWM:xxx` adalah bentuk di jalur serial yang dicetuskan oleh `set_pwm()`, sedangkan `p<val>` adalah bentuk yang diketikkan pengguna pada kotak input. Karena teks bantuan pada saat program berjalan mengumumkan bentuk `p<val>`, bentuk itulah yang harus dipakai, dan `PWM:220` ditolak sebagai masukan tidak sah. Ketidaksesuaian pada docstring tidak merusak jalannya program, tetapi berpotensi menyesatkan siapa pun yang membaca kode tanpa menjalankannya.



Gambar 7. Penolakan perintah PWM:220 oleh program Python

Masukan kosong, yang terbentuk ketika tombol Enter ditekan tanpa mengetik apa pun, jatuh ke cabang `else` yang sama. Setelah `input()` diikuti `.strip()`,

rentetan hasilnya menjadi kosong dan tidak cocok dengan satu pun cabang sebelumnya. Perulangan `while True` tidak terganggu, program tetap berada pada mode interaktif, dan pesan peringatan dicetak ulang. Pelajaran yang dapat diambil adalah bahwa validasi masukan sebaiknya tidak dijadikan satu-satunya pengaman, karena balasan dari ESP32 pun berpeluang terpotong. Fungsi `kirim_perintah()` menunggu 0.15 detik lewat `time.sleep(0.15)` sebelum mulai membaca, dan perulangan `while ser.in_waiting > 0` berhenti begitu buffer sementara kosong. Bila firmware mengeluarkan balasan yang panjang atau tertunda, sisa baris balasan dapat tidak terbaca dan hilang dari tampilan notebook.



Gambar 8. Penolakan masukan kosong oleh validasi program

Satu mekanisme perlindungan yang layak diapresiasi terletak pada blok `finally`. Ketika perulangan diakhiri, baik melalui perintah `q` maupun melalui `KeyboardInterrupt`, program lebih dahulu memanggil `motor_off(esp32)` di dalam blok `try` tersendiri sebelum menjalankan `esp32.close()`. Akibatnya motor tidak dibiarkan berputar pada nilai duty terakhir saat program dihentikan. Bila port serial seolah masih terpegang setelah program berhenti, urutan inilah yang menjadi penyebabnya, bukan kesalahan pengiriman perintah.

2.8 Rangkuman hasil pengujian

Tabel berikut menyatukan seluruh masukan yang dicoba beserta hasil yang tercatat.

No	Masukan	Keluaran tercatat	Kelakuan perangkat
1	1	>> Motor ON dan ESP32: Motor ON, duty = 200	Roda berputar, LED biru menyala
2	+	>> Kecepatan +10 dan ESP32: Kecepatan NAIK, duty = 210	Nilai duty naik satu langkah
3	+	>> Kecepatan +10 dan ESP32: Kecepatan NAIK, duty = 220	Nilai duty naik satu langkah lagi
4	-	>> Kecepatan -10 dan ESP32: Kecepatan TURUN, duty = 210	Nilai duty turun satu langkah
5	0	>> Motor OFF dan ESP32: Motor OFF, duty = 0	Roda berhenti, LED biru padam
6	PWM:220	[!] Perintah tidak valid! Gunakan: 1, 0, +, -, p<angka> (misal: p150), atau q.	Tidak ada perintah yang diteruskan ke ESP32
7	kosong	Pesan peringatan yang sama	Program bertahan pada mode interaktif

Dua pola menonjol dari rangkuman ini. Pertama, setiap masukan sah selalu menghasilkan sepasang baris, yaitu cetakan dari sisi Python dan balasan dari perangkat, sehingga pembacaan balasan dapat dipakai untuk memverifikasi bahwa perintah sampai ke tujuan. Kedua, kedua jenis masukan tidak sah berhenti di lapisan Python dan tidak pernah menyentuh jalur serial, sehingga program dapat

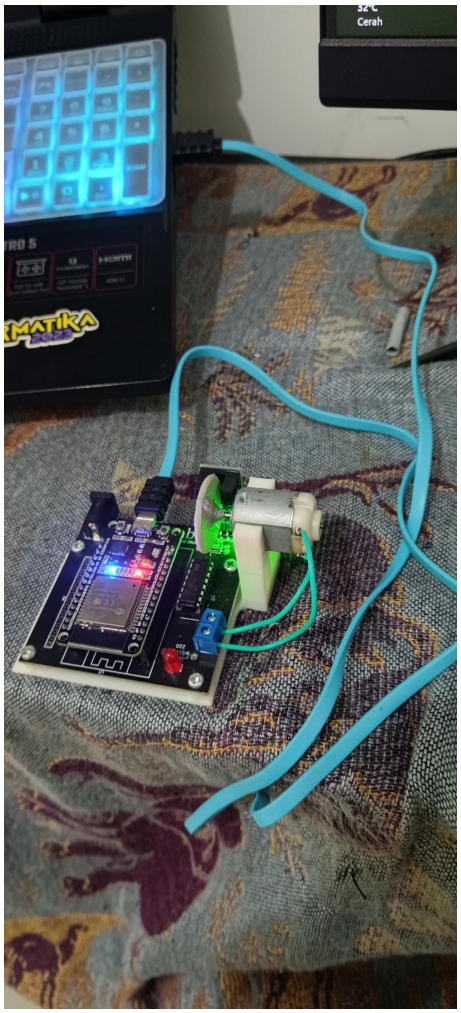
dikatakan menyaring masukan sebelum meneruskannya ke perangkat keras. Data yang

dipakai pada tabel ini seluruhnya berasal dari keluaran yang tercatat pada berkas

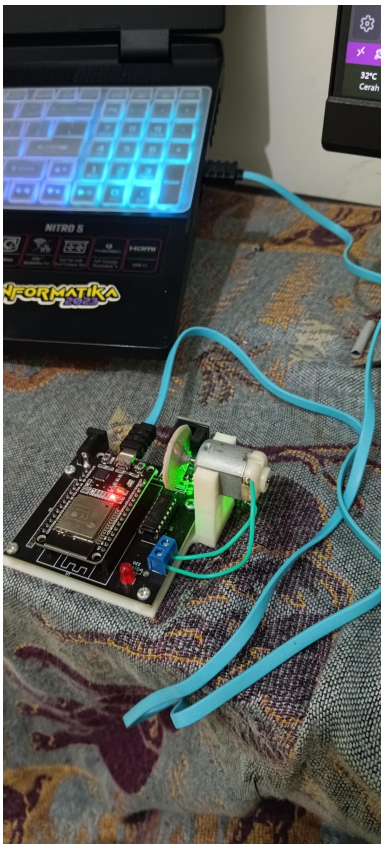
acuan dan tidak ditambahi angka lain.

2.9 Dokumentasi Praktikum

Praktikum modul kedua menggunakan kit iMCLab yang sama, tetapi perintah kontrol dikirim dari komputer melalui program Python pada Jupyter Notebook. Koneksi dibuka pada port serial hasil deteksi otomatis, sehingga roda dan LED pada kit bereaksi terhadap perintah yang dikirim dari komputer. Foto berikut mendokumentasikan keadaan kit selama pengujian berlangsung.



Gambar 9. Kit iMCLab saat dikendalikan melalui program Python pada Jupyter Notebook



Gambar 10. Kit iMCLab dan roda piringan selama pengujian antarmuka Python

BAB III PENUTUP

3.1 Kesimpulan

1. Pemilihan port serial berhasil dilakukan tanpa campur tangan pengguna. Fungsi `deteksi_port_esp32()` memberi bobot +100 pada pasangan VID:PID yang terdaftar,

   +50 pada kecocokan kata kunci deskripsi, dan −200 pada port Bluetooth, sehingga COM7 dengan identitas 10C4:EA60 terpilih pada skor akhir 150 dan koneksi terbuka

   pada 115200 baud.

2. Perintah `1`, `0`, `+`, dan `-` bekerja sesuai rancangan, dengan nilai duty

   bergerak 200 → 210 → 220 → 210 dan berakhir pada 0 saat motor dimatikan.

   Rentetan `q` menghentikan perulangan interaktif, sedangkan `Ctrl+C` ditangani

   melalui penangkapan `KeyboardInterrupt`.

3. Perubahan nilai duty berdampak nyata pada perangkat keras. Roda berputar dan

   LED biru pada GPIO 2 menyala saat `dutyCycle` bernilai 200, lalu berhenti dan

   padam saat nilai itu dijadikan 0.

4. Antarmuka Python memakai bentuk `p<val>` seperti diputuskan oleh percabangan `cmd.lower().startswith("p") and cmd[1:].isdigit()`. Akibatnya `PWM:220`

   ditolak meskipun bentuk itu sah pada Serial Monitor dan diterima firmware;

   bentuk yang benar dari sisi pengguna adalah `p220`.

5. Ketahanan program cukup baik. Blok `finally` memanggil `motor_off()` sebelum

   menutup port sehingga motor tidak dibiarkan berputar saat program dihentikan,

dan masukan kosong maupun perintah tidak sah ditangani tanpa mematikan sesi.

3.2 Saran

1. Docstring fungsi `kirim_perintah()` sebaiknya disamakan dengan antarmuka yang benar-benar dijalankan, yaitu `p<val>`, agar daftar perintah pada dokumentasi tidak bertentangan dengan teks bantuan yang tampil pada saat program dijalankan.
2. Pembacaan balasan dapat dibuat lebih tahan terhadap pemotongan, misalnya dengan menunggu sampai baris baru diterima atau menambah tenggang tunggu, sehingga balasan panjang dari firmware tidak hilang sebagian.
3. Pesan bantuan mode interaktif dapat dilengkapi penjelasan rentang nilai duty yang sah, yaitu 0 sampai 255, mengingat fungsi `set_pwm()` sudah melakukan pembatasan melalui `max(0, min(255, int(duty)))`.
4. Mekanisme cadangan pada `deteksi_port_esp32()` dapat diperluas. Saat ini port tunggal non-Bluetooth langsung dipakai hanya jika jumlahnya tepat satu; menampilkan pilihan kepada pengguna ketika port lebih dari satu akan membuat program lebih aman dipakai pada komputer dengan banyak perangkat serial.